

STCF — Shortest Time to Completion First

One CPU, whole ticks, CPU-only processes. Service is the CPU work needed to finish. Switching processes costs no time.

Rules

1. Choose the process with the shortest remaining service among the runner and ready processes.
2. Preempt the runner only when a ready process has **strictly less** remaining service. On equality, keep the current runner.
3. Append a preempted process to the ready-queue tail, preserving its remaining service and recorded metrics.
4. When dispatching from a free CPU, break equal-remainder ties by earlier arrival, then input order. This does not override Rule 2.
5. Each running tick reduces remaining service by one. At zero, the process leaves the system.
6. If no process is ready, wait for the next arrival. Stop when all processes finish.

The ready queue keeps insertion order; selection searches the whole queue. There is no quantum or aging rule.

At a time boundary

Complete the runner if finished → add arrivals → check for a strictly shorter ready process → dispatch if the CPU is free.

All same-time arrivals are considered before selection. New arrivals enter before a preempted runner is appended.

Example

Input order: A, B, C.

Process	Arrival	Service
A	0	5
B	1	2
C	1	1

CPU: A [0–1), C [1–2), B [2–4), A [4–8).

At time 1, A has four ticks left. C needs one and B needs two, so C preempts A. A later resumes its remaining four ticks.

Equal-time case: If A has three ticks left and B arrives needing three, A keeps running.